



Project Title: Air Quality Dashboard and Analysis

Submitted in partial fulfillment of the requirements for the award of the
degree of

Master of Computer Application (MCA)

By

Student Name: Shivam Singh

Enrollment No: O24MCA111082

Under the guidance of

Guide/Mentor Name: Vandana Sivaraj

Certificate

This page will include the certificate issued by Qollabb upon completion of the project.

Declaration

I, **Shivam Singh**, hereby solemnly declare that the project report titled "**Air Quality Analysis and Visualization Dashboard**" submitted in partial fulfillment of the requirements for the award of the degree of **Master of Computer Application (MCA)** is my original work.

I further declare that:

- This project has been carried out by me during the academic year 2026 under the supervision of **Vandana Sivaraj**
- The work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma, or certification.
- All sources of information used in this report have been duly acknowledged and referenced in accordance with academic ethics and plagiarism norms.
- The data presented in this report is authentic to the best of my knowledge and has not been fabricated or manipulated.
- I understand that if any part of this declaration is found to be false, my project may be rejected and disciplinary action may be taken as per institutional rules.

Place: CU university

Date: 15th may 2026

Student Signature: Shivam Singh

Student Name: Shivam Singh

Enrollment No: 024MCA111082

Acknowledgement

I would like to express my sincere gratitude to my project guide for their continuous support, valuable guidance, and encouragement throughout the development of this project. Their suggestions and feedback helped me improve my work at every stage.

I am also thankful to my department and faculty members for providing the necessary resources and knowledge required to complete this project successfully.

I would like to extend my thanks to my friends and classmates for their support and helpful discussions during the project work.

Finally, I am grateful to my family for their constant motivation and encouragement, which helped me stay focused and complete this project on time.

Abstract

This project aims to analyse and visualize air quality data using an interactive dashboard. The main objective is to study AQI levels of different cities and present the data in an easy and understandable format. The dataset includes AQI values and pollutants like PM_{2.5}, PM₁₀, NO₂, SO₂, CO, and O₃.

The system uses Python for data analysis, MySQL for storing the data, and Power BI for visualization. A simple web interface is also created using Flask to display AQI data and graphs based on user selection.

The project includes data collection, preprocessing, database storage, and visualization. The system was tested to ensure proper functioning and accurate results. The output helps users understand air pollution trends and compare different cities.

In future, the project can be improved by adding real-time data and more advanced analysis features.

Table of Contents

Contents

Certificate	1
Declaration	2
Acknowledgement.....	3
Abstract.....	4
Table of Contents.....	5
Abstract.....	6
Chapter 1: Introduction	7
Chapter 2: System Study.....	14
Chapter 3: System Analysis.....	22
Chapter 4: System Design.....	30
Chapter 5: System Implementation	43
Chapter 6: Testing	50
Chapter 7: Results & Discussion	55
Chapter 8: Conclusion & Future Scope	61
Chapter 9: References.....	64
Chapter 10: Appendices.....	68

Abstract

This project focuses on the analysis and visualization of air quality data using an interactive dashboard system. The main objective of the study is to understand air pollution trends across different cities and present the information in a simple and user-friendly manner. The dataset used in this project includes Air Quality Index (AQI) values and various pollutants such as PM_{2.5}, PM₁₀, NO₂, SO₂, CO, and O₃.

The system is developed using Python for data analysis, MySQL for database management, and Power BI for visualization. A web-based interface is also created using Flask, which allows users to select cities and view AQI data along with graphical representations. The project involves data collection, preprocessing, storage, and visualization to generate meaningful insights.

The system was tested to ensure correct data display and smooth functionality. The results help in understanding pollution patterns and comparing air quality across different locations. In the future, the system can be improved by integrating real-time data and advanced analytical features to enhance its effectiveness.

Chapter 1 : Introduction

1.1 Background of Air Pollution

Air pollution has become one of the most serious environmental challenges faced by modern society. Rapid urbanization, industrial growth, increasing vehicular emissions, and expanding energy consumption have contributed significantly to the deterioration of air quality in many parts of the world. Over the past few decades, the concentration of harmful pollutants in the atmosphere has increased considerably, affecting both human health and the natural environment.

In developing countries, the problem of air pollution is particularly severe due to rapid population growth and unplanned urban expansion. Major metropolitan cities experience high levels of pollutants released from automobiles, industrial facilities, construction activities, and burning of fossil fuels. These pollutants accumulate in the atmosphere and lead to poor air quality conditions, especially in densely populated urban areas.

India is among the countries facing significant challenges related to air pollution. Several cities frequently record high pollution levels, which pose serious risks to public health. In recent years, environmental agencies and government bodies have taken initiatives to monitor and control air pollution by collecting and analyzing air quality data from different regions. However, the availability of large volumes of environmental data also creates the need for effective methods to analyze and interpret this information.

Air quality monitoring systems play an important role in understanding pollution patterns and identifying the sources of environmental degradation. These systems collect data related to various pollutants present in the atmosphere and provide valuable insights into the condition of the air in different geographical locations. The analysis of such data helps researchers, policymakers, and environmental organizations develop strategies for pollution control and environmental protection.

1.2 Importance of Air Quality Monitoring

Air quality monitoring is an essential process for assessing the level of pollution present in the atmosphere. Monitoring systems measure the concentration of various pollutants such as particulate matter (PM_{2.5} and PM₁₀), nitrogen dioxide (NO₂), sulfur dioxide (SO₂), carbon monoxide (CO), and ozone (O₃). These pollutants originate from multiple sources including vehicle emissions, industrial processes, power generation plants, and natural events such as dust storms and forest fires.

The continuous monitoring of air quality allows environmental authorities to track pollution levels and detect any abnormal increases in pollutant concentrations. This information helps government agencies implement appropriate regulatory measures to reduce pollution and protect public health. Additionally, air quality monitoring provides essential data that can be used for research and environmental studies.

One of the key challenges in air quality monitoring is the effective interpretation of collected data. Large datasets generated from monitoring stations often require advanced analytical techniques to extract meaningful insights. Without proper analysis and visualization, raw environmental data may be difficult for decision-makers and the general public to understand.

Therefore, there is a growing need for systems that can analyze air quality data efficiently and present the results in a clear and interactive manner. Data analytics and visualization tools provide an effective solution for transforming complex datasets into meaningful information that can support environmental decision-making.

1.3 Air Quality Index (AQI)

The Air Quality Index (AQI) is a standardized indicator used to communicate the level of air pollution to the public. It provides a simplified representation of complex environmental data by converting pollutant concentrations into a single numerical value that indicates the overall air quality condition of a specific location.

The AQI scale typically ranges from values representing good air quality to levels that indicate hazardous pollution conditions. Different ranges of AQI values correspond to different health categories, helping individuals understand how air pollution may affect

their health. For example, lower AQI values generally indicate clean air with minimal health risks, while higher values suggest unhealthy air conditions that may cause respiratory problems and other health issues.

Environmental protection agencies around the world use AQI as a standard method to inform the public about daily air quality conditions. Governments often publish AQI reports to help people take necessary precautions during periods of high pollution. For instance, individuals with respiratory diseases may avoid outdoor activities when the AQI level indicates unhealthy conditions.

Because AQI combines multiple pollutant measurements into a single index, it provides a convenient way to compare air quality levels across different cities and time periods. This makes AQI an important tool for environmental analysis and public awareness.

1.4 Need for Data Analysis and Visualization

With the advancement of technology, large volumes of environmental data are being generated from monitoring stations, satellites, and research institutions. Analysing this data manually is both time-consuming and inefficient. As a result, data analytics techniques have become increasingly important in environmental studies.

Data analysis helps researchers identify patterns, trends, and relationships within air quality datasets. By analyzing historical data, it becomes possible to understand how pollution levels change over time and how different factors influence air quality conditions.

However, raw data tables alone are often difficult to interpret. Visualization techniques such as charts, graphs, and dashboards allow complex information to be presented in a more understandable form. Interactive dashboards enable users to explore datasets dynamically and gain insights into environmental trends without requiring advanced technical knowledge.

Visualization tools also play a significant role in communicating environmental information to policymakers and the general public. Clear visual representations of air

quality trends can support informed decision-making and promote awareness about pollution issues.

1.5 Role of Data Analytics in Environmental Monitoring

Advancements in technology have led to the generation of large volumes of environmental data from monitoring stations, satellites, and research institutions. Analyzing this data manually is both time-consuming and inefficient. As a result, data analytics has become an essential component of modern environmental research.

Data analytics techniques allow researchers to process large datasets and extract meaningful insights. By analyzing historical air quality data, it becomes possible to identify patterns, detect anomalies, and understand the factors that influence pollution levels. These insights help researchers develop models that can predict future pollution trends and evaluate the effectiveness of environmental policies.

In addition to traditional statistical methods, modern data analytics tools enable interactive exploration of datasets. Visualization techniques such as charts, graphs, and dashboards make it easier to interpret complex information and communicate findings to a wider audience. These tools transform raw data into visual representations that highlight key trends and relationships within the data.

1.6 Need for Data Visualization and Dashboards

Data visualization plays a critical role in transforming complex datasets into understandable information. While raw data tables may contain valuable information, they are often difficult to interpret without visual representation. Charts, graphs, and dashboards provide a more intuitive way of analyzing data and identifying patterns.

Interactive dashboards have become widely used in various fields such as business analytics, healthcare, finance, and environmental monitoring. These dashboards allow users to explore datasets dynamically, apply filters, and view data from different perspectives. By presenting information in a visually appealing format, dashboards improve the accessibility and usability of data.

In the context of environmental monitoring, visualization dashboards can help researchers and policymakers quickly identify pollution trends, compare air quality across different cities, and evaluate the effectiveness of pollution control measures. Such systems also improve public awareness by presenting environmental information in a clear and understandable way.

1.7 Motivation for the Project

The increasing concern about air pollution and its impact on public health has motivated the development of tools that can effectively analyse and visualize environmental data. While large datasets related to air quality are available, there is often a lack of user-friendly systems that allow individuals to explore this data interactively.

The motivation behind this project is to develop a system that integrates data analysis, database management, and visualization techniques to provide meaningful insights into air quality trends. By creating an interactive dashboard, users can easily analyse pollution data, compare air quality levels across cities, and understand how pollution patterns change over time.

Such a system can support environmental awareness and help individuals gain a better understanding of air quality conditions in different regions.

1.8 Problem Statement

Despite the availability of air quality datasets, analyzing and interpreting this data can be challenging without appropriate tools. Large datasets stored in databases often require technical expertise to query and analyse, making it difficult for non-technical users to extract meaningful insights.

Furthermore, many traditional reporting systems rely on static data presentations that do not allow users to interact with the data. This limits the ability to explore different aspects of air quality trends and compare pollution levels across locations.

Therefore, there is a need for an integrated system that can process air quality data efficiently and present the results through interactive visualizations and dashboards.

1.9 Objectives of the Project

The main objectives of this project are:

- To analyze air quality data collected from different cities.
- To store and manage environmental data using a relational database system.
- To develop data processing and analysis modules using Python.
- To create interactive visualizations that display AQI trends and pollutant levels.
- To design a user-friendly dashboard interface that allows users to explore air quality data easily.

1.10 Scope of the Project

- The scope of this project focuses on the analysis and visualization of air quality data using modern data analytics tools. The system integrates multiple technologies, including Python for data processing, MySQL for database management, and Power BI for advanced data visualization.
- In addition, a web-based interface is developed to allow users to interact with the dataset and view graphical representations of pollution trends. The system enables users to select different cities and analyse AQI trends through dynamic charts and tables.
- Although the project focuses on selected cities and datasets, the system design can be extended to include additional data sources and more advanced analytical features in the future.

1.11 Organization of the Report

This project report is organized into several chapters to provide a clear understanding of the work carried out in this study.

Chapter 1 introduces the background of air pollution, the importance of air quality monitoring, and the motivation behind the project.

Chapter 2 presents the literature review, which discusses previous research and studies related to air quality analysis and visualization.

Chapter 3 describes the system design and architecture of the proposed solution.

Chapter 4 explains the implementation process, including the technologies and tools used for developing the system.

Chapter 5 presents the results and analysis obtained from the developed dashboard.

Finally, Chapter 6 summarizes the conclusions of the project and suggests possible future improvements.

Chapter 2: System Study

2.1 Existing System

In the present time, air quality monitoring is mainly done through government websites and a few online platforms. These systems provide Air Quality Index (AQI) data for different cities. However, most of these platforms are not very user-friendly and are often difficult to understand for general users.

Many existing systems display data in a static format, such as tables or simple charts, without much interaction. Users cannot easily filter data based on their needs, such as selecting a specific city or analyzing trends over time. Also, some websites are overloaded with information, making it confusing for beginners.

Another issue is that many systems do not provide clear visualization of data. While the data is available, it is not presented in a way that helps users quickly understand air quality conditions.

2.2 Problems in Existing System

There are several limitations in the existing systems:

- Most platforms lack interactive features for users
- Data is often displayed in a complex format
- Difficult to compare AQI values between cities
- Lack of proper graphical representation
- Not suitable for beginners or non-technical users
- Limited customization options
- Slow or cluttered user interfaces

Because of these issues, users may find it difficult to understand and analyze air quality data effectively.

2.3 Proposed System

To overcome the limitations of the existing system, this project proposes an **Air Quality Analysis and Visualization Dashboard**.

This system is a web-based application where users can select a city and view its AQI data in both table and graphical formats. In addition to single city analysis, the system also provides a comparison feature that allows users to select two cities and analyze their AQI data together. This helps users to better understand differences in pollution levels. The main aim of this system is to make air quality data easy to understand and accessible to everyone.

The system allows users to:

- Select a city from a dropdown menu
- View AQI data in a structured table
- Visualize AQI trends using graphs
- Compare AQI levels between two cities
- View average AQI values for selected cities
- Analyze pollution differences using graphical comparison

This makes the analysis simple and more interactive compared to existing systems.

2.4 Advantages of Proposed System

The proposed system has several advantages:

- Simple and user-friendly interface
- Interactive data visualization using graphs
- Easy selection of cities
- Faster data access and display
- Helps users understand AQI trends clearly
- Lightweight and efficient system

This system improves the overall user experience and makes data analysis easier.

2.5 Feasibility Study

a) Technical Feasibility

The project is technically feasible as it uses widely available technologies such as Python, Flask, MySQL, Pandas, and Plotly. These technologies are easy to use, well-supported, and suitable for developing data-driven web applications. The system also supports additional functionalities such as comparison between two cities and calculation of average AQI, which can be efficiently handled using these tools..

b) Economic Feasibility

This project does not require any expensive tools or software. All technologies used are free or open-source, making it cost-effective. The system can be developed and deployed with minimal resources.

c) Operational Feasibility

The system is easy to operate and does not require any special training. Users can select a single city or compare two cities to view AQI data, graphs, and average pollution levels. The simple interface ensures smooth user interaction and ease of use.

2.6 Technologies Used

Python

Python is used as the main programming language for the project. It helps in data processing and backend development.

Flask

Flask is a lightweight web framework used to build the web application and handle user requests.

MySQL

MySQL is used as the database to store AQI data for different cities.

Pandas

Pandas is used for handling and processing data efficiently.

Plotly

Plotly is used to create interactive graphs for data visualization.

HTML/CSS

HTML and CSS are used to design the frontend of the application and make it visually appealing

2.7 System Requirements

Hardware Requirements

- Computer or laptop
- Minimum 4GB RAM

Software Requirements

- Python
- MySQL Database
- Code Editor (VS Code / PyCharm)
- Web Browser (Chrome, Edge, etc.)

2.8 System Architecture and Working

The system works in a simple and structured way:

1. The user opens the web application
2. The user logs in using valid credentials
3. The user selects either a single city or two cities for comparison from the dropdown menu
4. The request is sent to the Flask server
5. The server fetches AQI data from the MySQL database based on the selected input
6. Data is processed using Pandas (filtering, sorting, and average calculation)
7. A graph is generated using Plotly (single-city trend or comparison graph)
8. The system displays:
 - AQI data in table format (single or side-by-side tables)
 - Graphical visualization
 - Average AQI values (in case of comparison)
9. The results are displayed on the dashboard

This workflow ensures smooth functioning, efficient data processing, and an interactive user experience.

Chapter 3: System Analysis

3.1 Introduction

System Analysis is an important phase in the development of any project. In this phase, the requirements of the system are studied and analysed in detail. It helps in understanding what the system should do and how it should perform.

In this project, the system analysis focuses on understanding how air quality data can be collected, processed, and displayed in a meaningful way. The aim is to design a system that is simple, efficient, and useful for users.

3.2 Requirement Analysis

Requirement analysis is the process of identifying what the system needs to perform its tasks properly. It includes both user requirements and system requirements.

User Requirements

- The system must connect to a database
- It should fetch data based on user input (single or multiple cities)
- It must process data efficiently using Pandas
- It should calculate average AQI values for comparison
- It should generate graphs dynamically using Plotly
- It should support multiple routes (dashboard and comparison) in Flask **System**

Requirements

- The system must connect to a database
- It should fetch data based on user input (single or multiple cities)
- It must process data efficiently using Pandas
- It should calculate average AQI values for comparison
- It should generate graphs dynamically using Plotly
- It should support multiple routes (dashboard and comparison) in Flask

3.3 Functional Requirements

Functional requirements describe what the system should do.

- The system should allow selection of a single city
- The system should allow selection of two cities for comparison
- It should fetch AQI data from the database based on user input
- It should display AQI data in tabular format (single or multiple cities)
- It should generate graphical representations of AQI using Plotly
- It should calculate and display average AQI values for selected cities
- It should compare AQI levels and indicate which city is more polluted
- It should update data dynamically when a new city or cities are selected

These functions ensure that the system works properly and meets user requirements effectively.

3.4 Non-Functional Requirements

Non-functional requirements define how the system performs.

- The system should be fast and responsive
- It should be easy to use
- The system should be reliable
- It should handle data efficiently
- The design should be simple and clean

These requirements improve the overall quality of the system.

3.5 Data Analysis

In this project, AQI data is the most important part. The data is stored in a MySQL database and includes fields such as date, city, and AQI value.

The data is analyzed using the Pandas library in Python. It helps in reading, filtering, and organizing the data.

The system selects data based on the city chosen by the user. After that, it processes the data and prepares it for display in both table and graph formats.

This step is important because it ensures that the data shown to the user is accurate and meaningful.

3.6 Process Analysis

Process analysis describes how the system works step by step.

1. The user opens the application
2. The user logs in with valid credentials
3. The user selects either a single city or two cities for comparison
4. The request is sent to the Flask server
5. The server processes the request based on the selected option
6. Data is fetched from the MySQL database
7. Data is processed using Pandas
8. AQI trends are visualized using Plotly graphs
9. In comparison mode, average AQI values are also calculated
10. The results are displayed on the webpage in the form of tables, graphs, and comparison output

This process ensures smooth functioning of the system and provides an interactive user experience.

3.7 Input Design

Input design focuses on how the user provides data to the system.

In this project, input is taken through a dropdown menu. The user selects a city from the list provided.

The input method is simple and reduces the chances of errors. It also improves user experience by making the system easy to use.

3.8 Output Design

Output design defines how the system presents information to the user.

The system provides output in two formats:

- Table format for detailed data
- Graph format for visual understanding

The table shows AQI values along with dates, while the graph shows trends over time.

This makes it easier for users to understand the air quality condition.

3.9 System Modeling

System modeling helps in representing the system visually. It shows how different parts of the system interact with each other.

In this project, the system includes:

- User Interface
- Flask Server
- Database
- Data Processing Module

All these components work together to provide the final output.

You can also include diagrams like:

- Data Flow Diagram (DFD)
- System Architecture Diagram

3.10 Constraints of the System

Every system has some limitations. The constraints of this system are:

- Limited to selected cities only
- Depends on availability of data
- Requires internet and system setup
- Basic level visualization

These constraints can be improved in future versions.

3.11 Summary

System Analysis helps in understanding the requirements and structure of the project. It ensures that the system is properly planned before implementation.

In this project, system analysis plays a key role in designing an efficient and user-friendly AQI dashboard. It provides a clear idea of how the system will work and what features it will include.

Chapter 4: System Design

4.1 Introduction

System Design is an important phase in software development where the overall structure of the system is planned. It defines how different components of the system interact with each other and how the system will function.

In this project, the system is designed to analyze and visualize air quality data using a web-based dashboard. The design ensures that the system is simple, efficient, and user-friendly while providing meaningful insights through data visualization.

4.2 Design Objectives

The main objectives of system design are:

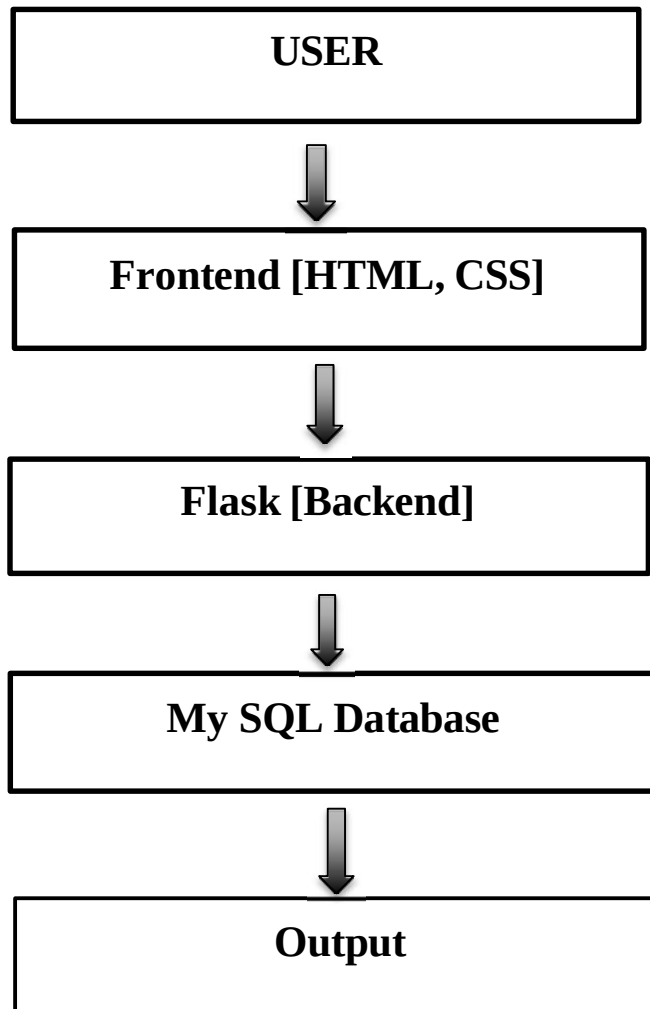
- To create a simple and user-friendly interface
- To ensure efficient data processing
- To provide accurate AQI data visualization
- To design a scalable system
- To maintain data consistency and reliability

4.3 System Architecture

The system follows a three-tier architecture:

1. Presentation Layer (Frontend) – HTML, CSS
2. Application Layer (Backend) – Flask
3. Data Layer (Database) – MySQL

These layers interact with each other to process user requests and display results.



(Fig: System Architecture Diagram)

4.4 System Workflow

The workflow of the system is as follows:

1. User opens the web application
2. User logs in using valid credentials
3. User selects either a single city or two cities for comparison
4. Request is sent to the Flask server
5. Data is fetched from the MySQL database based on the selected input
6. Data is processed using Pandas (filtering and average calculation)
7. Graph is generated using Plotly (single-city trend or comparison graph)
8. Results are displayed on the webpage, including tables, graphs, and average AQI values

This workflow ensures efficient data processing, smooth interaction, and better understanding of air quality data.

4.5 Data Flow Diagram (DFD)

Data Flow Diagram (DFD) represents how data flows through the system. It shows how input is processed and converted into output.

4.5.1 DFD LEVEL 0

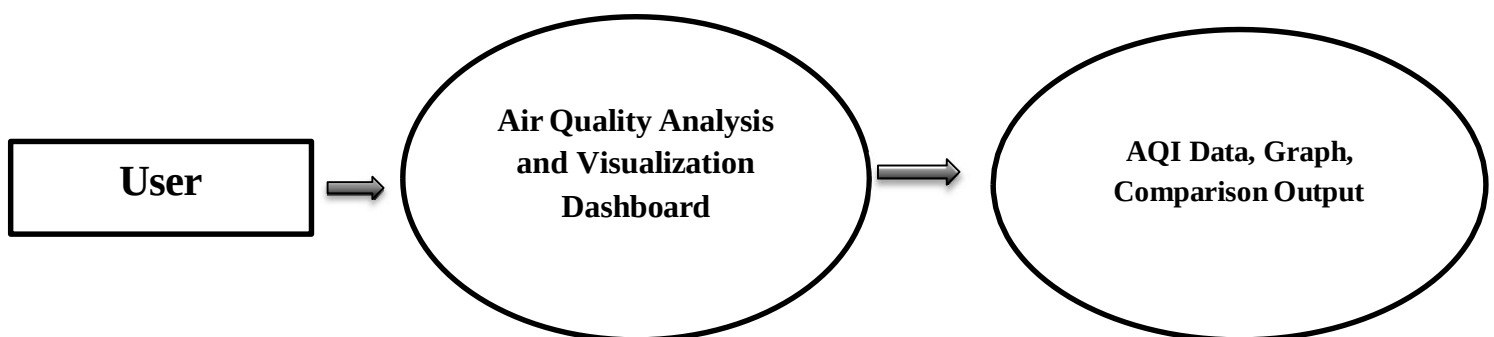


Figure 4.2: DFD Level 0

4.5.2 DFD LEVEL 1

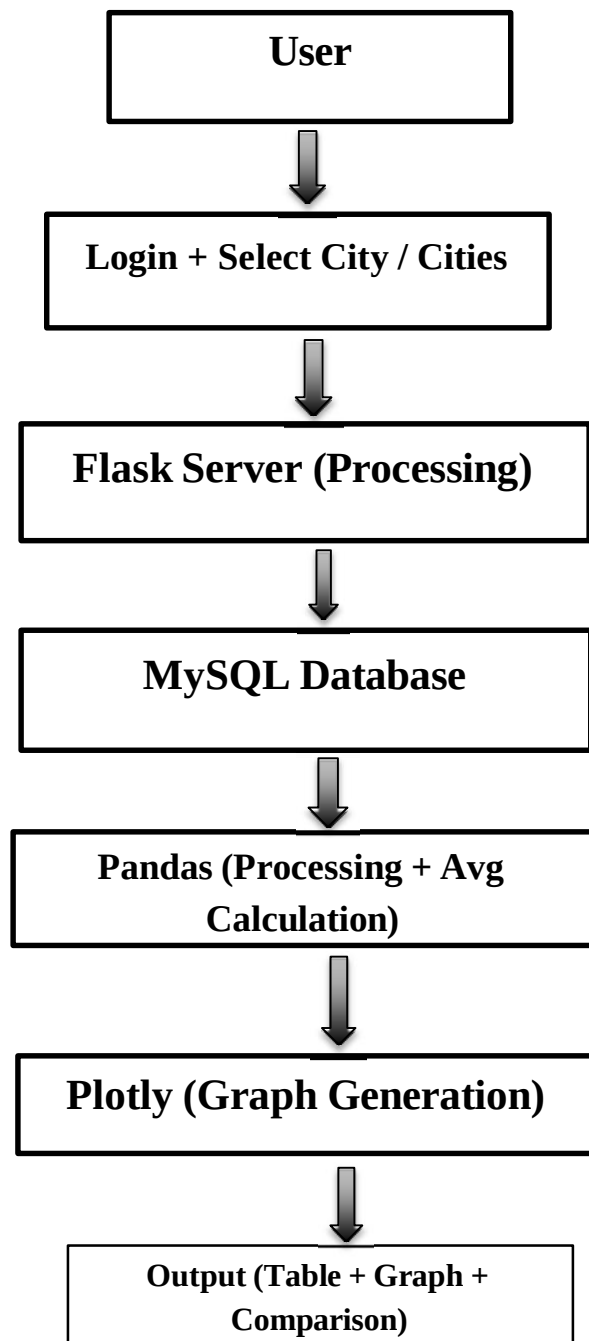


Figure 4.3: DFD Level 1

4.6 Database Design

The database is designed to efficiently store and manage air quality data. It is structured into multiple tables to ensure better organization, reduce redundancy, and improve scalability.

4.6.1 Table: Cities

Field Name	Data Type	Description
city_id	INT (PK)	Unique id key
city_name	VARCHAR	Name of city

- Purpose: Stores list of cities

4.6.2 Table: Air_quality_data

Field Name	Data Type	Description
id	INT (PK)	Unique record ID
city_id	INT (FK)	Reference cities Table
date	Date	Date of Record
aqi	INT	AQI value

- Purpose: Stores AQI data

4. Table: Users

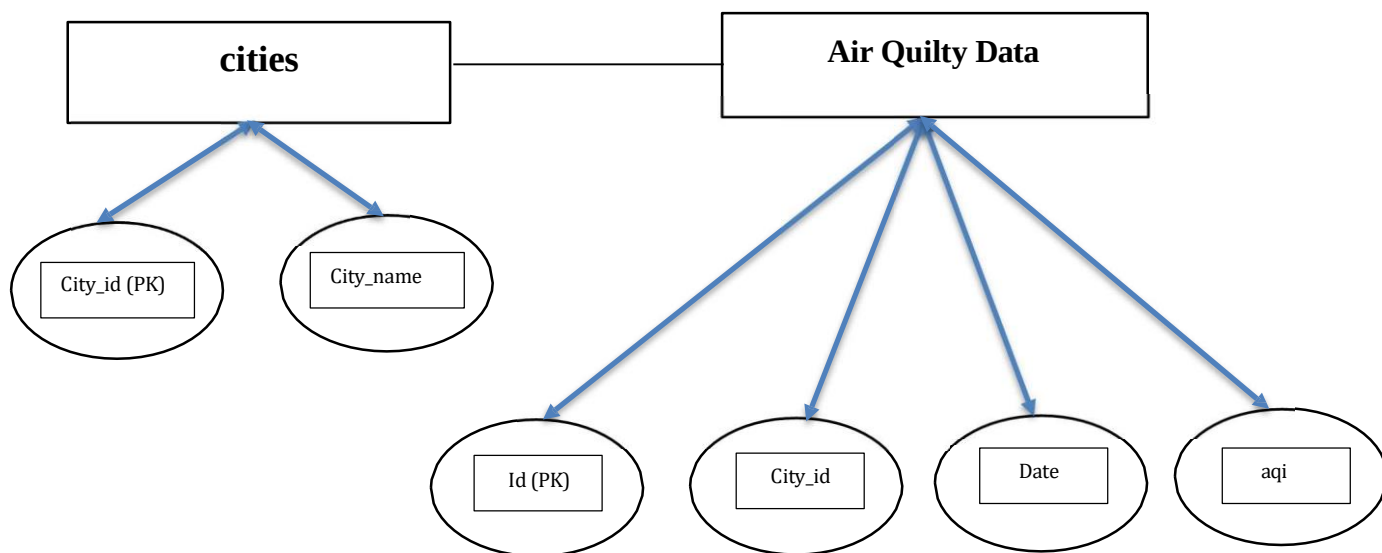
Field Name	Data Type	Description
userid	INT (PK)	Unique user ID
username	VARCHAR	User name
password	VARCHAR	User Password

➤ Purpose: For future login system

The database is normalized into multiple tables such as Cities and Air_Quality_Data to improve data organization and avoid redundancy. Each AQI record is linked to a specific city using a foreign key relationship.

4.7 ER Diagram (Entity Relationship Diagram)

The ER Diagram shows the relationship between entities.



4.8 Input Design

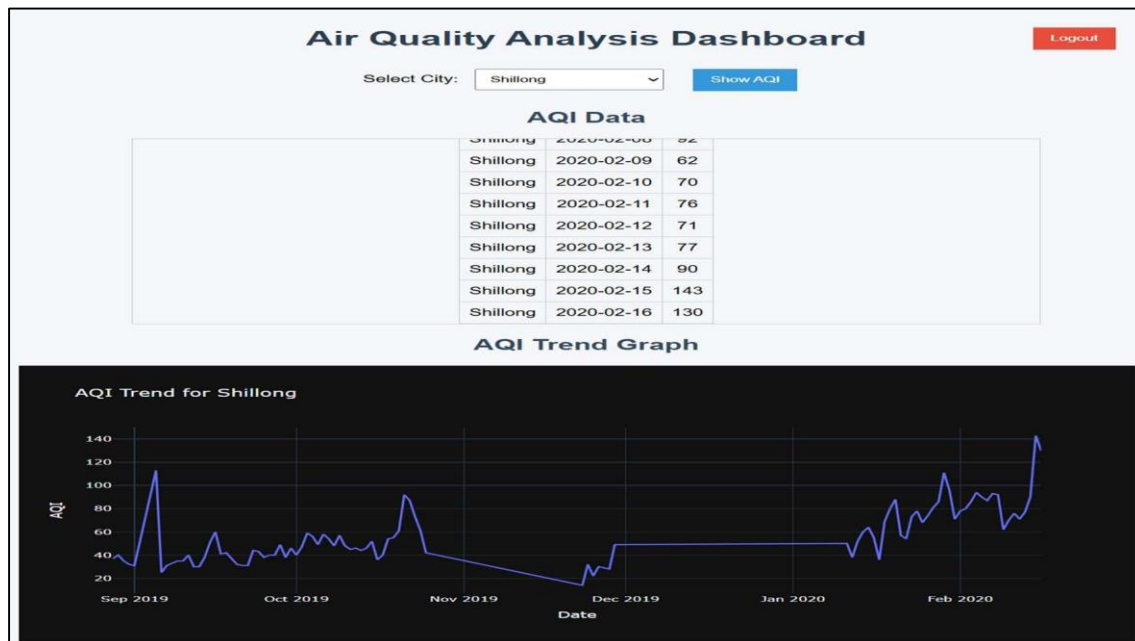
Input design focuses on how users provide data to the system.

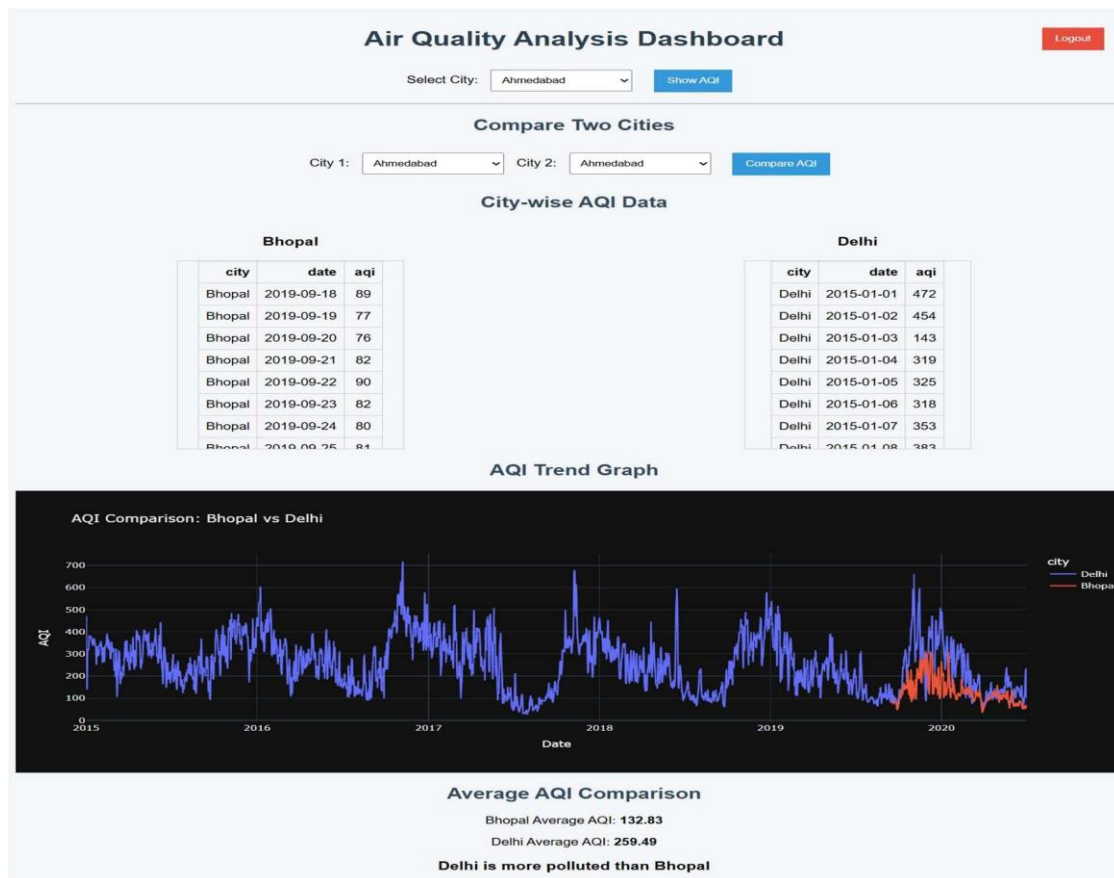
In this project, input is taken through dropdown menus. The system provides two types of input options:

- Single city selection using a dropdown menu
- Selection of two cities for comparison using two dropdown menus

These dropdowns help prevent invalid input and ensure that users select only available cities from the database.

The input design is simple, user-friendly, and reduces the chances of errors while improving overall usability.





4.9 Output Design

The output design defines how the system presents information to the user in a clear and understandable format.

In this project, the output is displayed in multiple forms to improve data understanding:

- **Tabular Format:**

The AQI data is displayed in a structured table showing city, date, and AQI values. In comparison mode, the data is displayed in two separate tables side-by-side for better clarity.

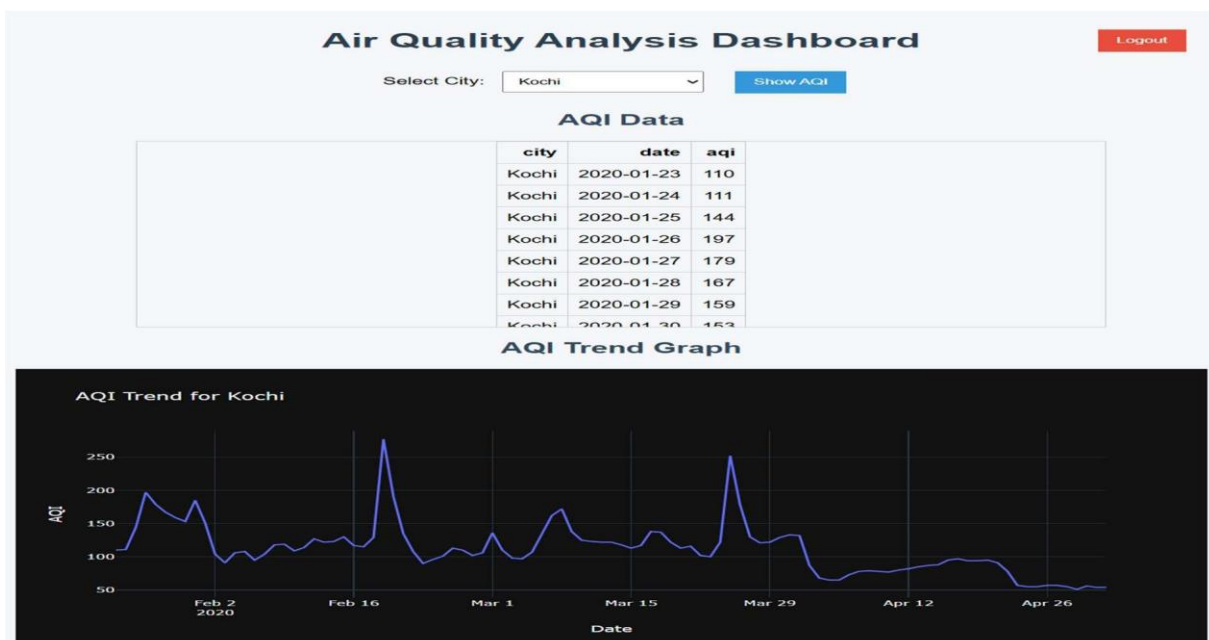
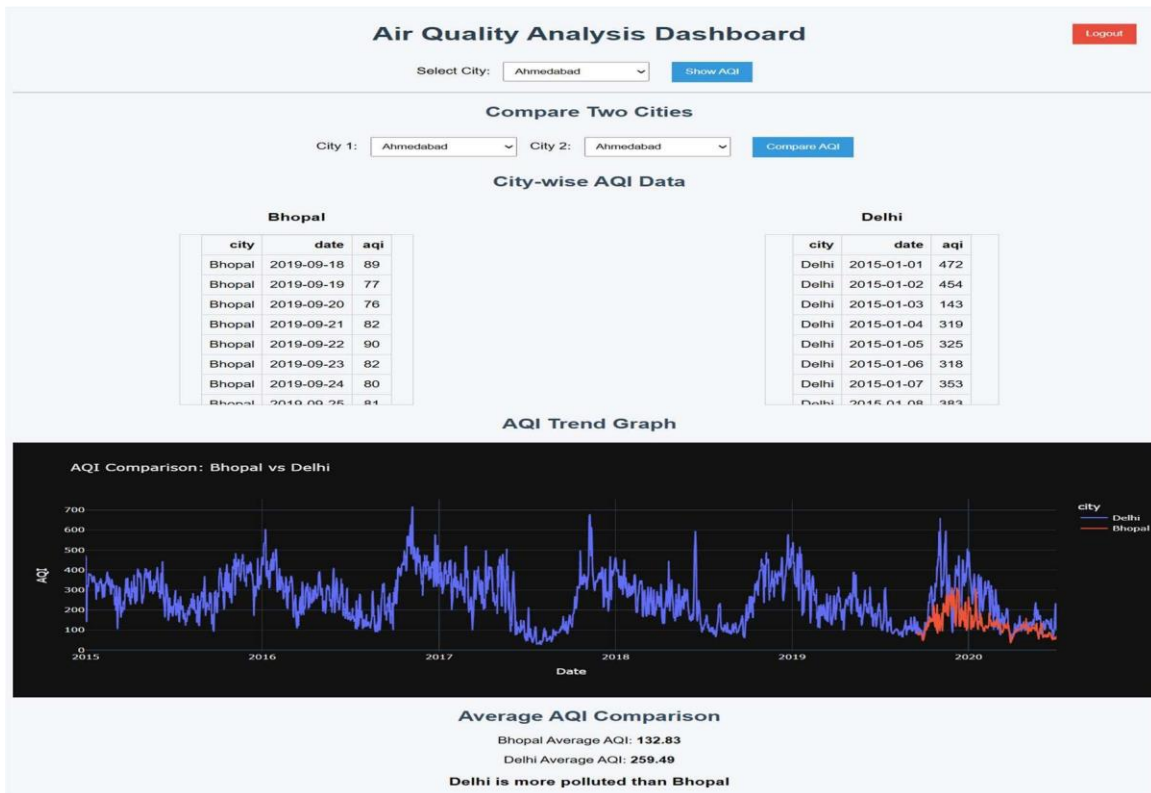
- **Graphical Format:**

An interactive graph is generated using Plotly to visualize AQI trends over time. In comparison mode, the graph displays multiple lines representing different cities.

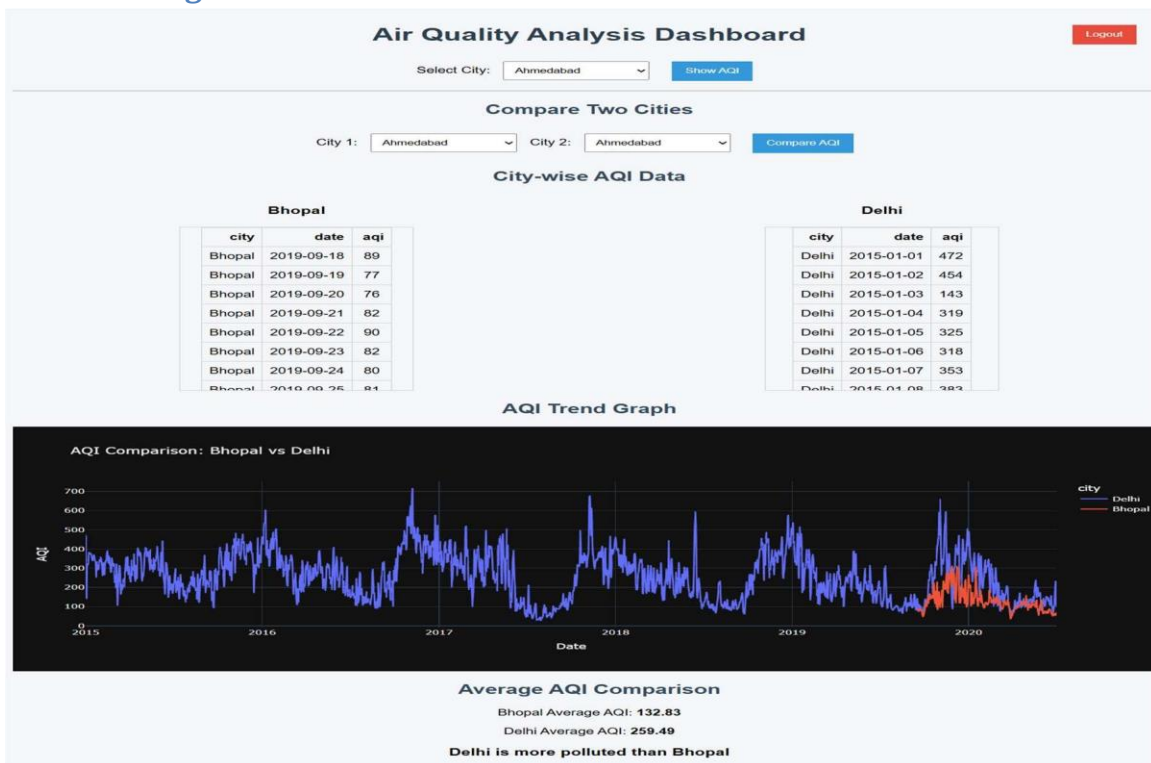
- **Comparative Analysis:**

The system provides comparison results between two selected cities. It calculates and displays the average AQI values for each city and indicates which city is more polluted.

The combination of tables, graphs, and comparison results improves readability, enhances user experience, and provides deeper insights into air quality data.



4.10 UI Design



Login

The User Interface (UI) of the system is designed to be simple, clean, and user-friendly.

The dashboard includes:

- A dropdown menu for single city selection
- Two dropdown menus for selecting cities for comparison
- Buttons to fetch AQI data and compare cities
- A scrollable table to display AQI records
- Side-by-side tables for comparing data of two cities
- An interactive graph for visualization (single and comparison mode)
- Display of average AQI values and comparison result
- A logout button for user control

The design ensures easy navigation, clear data presentation, and a better user experience. It allows users to easily analyze and compare air quality data in an interactive manner.

4.11 Module Design

The system is divided into different modules for better organization and functionality.

- **Login Module:** Handles user authentication
- **Dashboard Module:** Displays AQI data and graphs
- **Database Module:** Stores and retrieves AQI data
- **Data Processing Module:** Uses Pandas to process data
- **Visualization Module:** Generates graphs using Plotly

Each module works together to ensure smooth functioning of the system.

4.12 Security

Security is an important aspect of the system.

- User authentication is implemented through login credentials
- Only authorized users can access the dashboard

- Data is securely stored in the MySQL database
- Basic validation is applied to prevent incorrect input

In future, security can be improved using password encryption and session management.

4.13 Error Handling

Error handling ensures that the system works smoothly even when issues occur.

- Database connection errors are handled properly
- Empty data or invalid city selection is managed
- The system prevents crashes by handling exceptions in Flask
- This improves system reliability and user experience.

Invalid login credentials show error messages

4.14 Future Scope

The system can be further improved with advanced features such as:

- Integration of real-time AQI data
- Adding more cities and pollutants
- Advanced filters like date range selection
- Improved UI using frameworks like Bootstrap
- Deployment on cloud platforms

These enhancements will make the system more powerful and scalable.

4.15 Summary

This chapter explained the system design of the Air Quality Dashboard. It covered input and output design, UI structure, module design, and system features like security and error handling.

The design ensures that the system is user-friendly, efficient, and capable of providing meaningful insights through data visualization.

chapter 5: System Implémentation

5.1 Introduction

The System Implementation phase is the stage where the proposed design is converted into a fully functional system. In this project, the Air Quality Analysis and Visualization Dashboard is developed using modern web technologies and data processing tools.

The implementation focuses on building a user-friendly web application that allows users to log in, select a city, and view AQI data in both tabular and graphical formats. The system integrates backend logic, database connectivity, and frontend design to provide a smooth and interactive experience.

5.2 Development Environment

The project is developed using the following tools and technologies:

- Programming Language: Python
- Web Framework: Flask
- Database: MySQL
- Frontend Technologies: HTML, CSS
- Libraries Used: Pandas, Plotly
- IDE: Visual Studio Code
- Operating System: Windows

These technologies were selected because they are lightweight, easy to use, and suitable for building data-driven web applications.

5.3 System Architecture Implementation

The system follows a three-tier architecture:

1. Presentation Layer (Frontend):
Handles user interaction through web pages designed using HTML and CSS.
2. Application Layer (Backend):
Implemented using Flask, which processes user requests and communicates with the database.

3. Data Layer (Database):

MySQL database stores AQI data and user credentials.

All three layers work together to ensure efficient data flow and proper system functionality.

5.4 Module Implementation

The system is divided into different modules for better organization and development.

5.4.1 Login Module

This module is responsible for user authentication.

- Users enter username and password
- Credentials are verified from the database
- On successful login, user is redirected to dashboard
- Invalid login shows error message

This ensures that only authorized users can access the system.

5.4.2 Dashboard Module

The dashboard is the main interface of the application. It provides users with interactive features to analyze and compare air quality data.

Features include:

- Single city selection dropdown
- Two dropdown menus for selecting cities for comparison
- Button to fetch AQI data
- Button to compare AQI between two cities
- Display of AQI data in tabular format
- Side-by-side tables for comparing data of two cities
- Interactive graph visualization (single-city and comparison mode)
- Display of average AQI values for selected cities
- Result indicating which city is more polluted
- Logout option

The dashboard is designed to provide a clear, interactive, and user-friendly experience, allowing users to easily analyze and compare air quality data.

5.4.3 Database Module

This module manages data storage and retrieval.

- Stores AQI data in structured format
- Maintains user login details
- SQL queries to fetch required data

Example operations:

- SELECT AQI data based on city
- Validate user credentials

5.4.4 Data Processing Module

The data processing module uses the Pandas library.

Functions:

- Reads data from database
- Filters data based on selected city
- Sorts and organizes data
- Prepares data for visualization

This module ensures that the data is accurate and ready for display.

5.4.5 Visualization Module

This module uses Plotly to generate graphs.

- Creates interactive line charts
- Displays AQI trends over time
- Improves understanding of pollution patterns

The graph updates dynamically based on selected city.

5.5 Database Implementation

The system uses MySQL database with the following tables:

1. users

- user_id (Primary Key)
- username
- password

2. air_quality_data

- id (Primary Key)
- city
- date
- aqi

The database is designed to store and retrieve data efficiently.

5.6 Implementation Steps

The development of the system is carried out in the following steps:

1. Setup Flask environment
2. Create database and tables in MySQL
3. Establish connection between Flask and MySQL
4. Develop login functionality
5. Design dashboard UI using HTML/CSS
6. Fetch data from database
7. Process data using Pandas
8. Generate graphs using Plotly
9. Display results on web page

10. Implement logout functionality

5.7 Code Structure

The project follows a well-organized folder structure to separate different components such as data, backend logic, database scripts, and frontend files.

Contains Power BI dashboard file for additional visualization.

📁 **Data/**

Stores CSV datasets used for analysis, such as:

- average_aqi_by_city.csv
- city_day.csv
- pollutant_comparison.csv
- delhi_shillong_comparison.csv

📁 **Database/**

Contains SQL scripts:

- queries.sql (used to create tables and fetch data)

📁 **Python_analysis/**

Contains Python scripts for data processing:

- aqi_analysis.py
- load_to_mysql.py
- dataset files used for preprocessing

📁 **web_app/**

Main Flask application folder:

- **app.py** → Backend logic
- **templates/** → HTML files
 - index.html
 - login.html
- **statics/** → CSS files
 - style.css

This structured approach helps in:

- - Separating frontend, backend, and data layers
- Making the project easy to understand and maintain
- Improving scalability for future enhancements

The modular structure ensures that each component of the system can be updated independently without affecting the entire application.

5.8 User Interface Implementation

The UI is implemented using HTML and CSS.

Features:

- Clean layout
- Dropdown for city selection
- Scrollable table
- Graph display area
- Logout button

The design focuses on simplicity and usability.

5.9 Integration of Components

All system components are integrated as follows:

- User interacts with frontend
- Request is sent to Flask backend
- Backend fetches data from database
- Data is processed using Pandas
- Graph is generated using Plotly

- Output is displayed to user

This integration ensures smooth data flow.

5.10 Challenges Faced During Implementation

Some challenges faced during implementation include:

- Column name mismatch (Date vs date)
- Database structure confusion
- Graph not displaying initially
- UI alignment issues
- Login integration problems

These issues were resolved through debugging and testing.

5.11 Improvements Made

The following improvements were made:

- Fixed column name errors
- Added login system
- Improved UI with scrollable table
- Added logout functionality
- Organized code structure

5.12 Summary

This chapter explained the implementation of the Air Quality Dashboard. It covered system architecture, module development, database implementation, and integration of different components.

The system was successfully implemented using Flask, MySQL, Pandas, and Plotly, resulting in a fully functional and interactive web application.

Chapter 6: Testing

6.1 Introduction

Testing is an important phase in software development that ensures the system works correctly and meets user requirements. It helps identify errors, bugs, and issues in the system before final deployment.

In this project, testing was performed to verify the functionality of the Air Quality Analysis and Visualization Dashboard. Different types of testing were carried out to ensure that all modules work properly and provide accurate results.

6.2 Objectives of Testing

The main objectives of testing are:

- To ensure the system works as expected
- To identify and fix errors
- To verify data accuracy
- To ensure smooth user interaction
- To check system performance

6.3 Types of Testing Performed

6.3.1 Functional Testing

Functional testing checks whether each feature of the system works correctly.

The following functionalities were tested:

- User login system
- City selection dropdown

- Fetching AQI data
- Display of table and graph
- Logout functionality

All features were tested with valid and invalid inputs.

6.3.2 Integration Testing

Integration testing ensures that different components of the system work together.

In this project:

- Flask backend was tested with MySQL database
- Pandas was tested with database data
- Plotly was integrated for graph generation

All components were successfully integrated.

6.3.3 User Interface Testing

UI testing ensures that the interface is user-friendly and works properly.

The following were tested:

- Layout design
- Dropdown functionality
- Button clicks
- Table scrolling
- Graph display

The interface was found to be simple and easy to use.

6.3.4 Database Testing

Database testing ensures correct storage and retrieval of data.

- Checked if AQI data is stored correctly

- Verified SQL queries
- Tested data retrieval based on city selection

All database operations were working correctly.

6.3.5 Performance Testing

Performance testing checks system speed and responsiveness.

- Data loading time was tested
- Graph generation speed was checked
- System response time was observed

The system performs efficiently under normal conditions.

6.4 Test Cases

The following test cases were used to verify system functionality:

Test Case ID	Test Description	Input	Expected Output	Status
TC01	Login with valid credentials	Correct username & password	Redirect to dashboard	Pass
TC02	Login with invalid credentials	Wrong username/password	Error message	Pass
TC03	Select city	Choose city from dropdown	AQI data displayed	Pass
TC04	Display table	Valid data	Table shown correctly	Pass
TC05	Display graph	Valid data	Graph displayed	Pass

TC06	Logout	Click logout	Redirect to login page	Pass
TC07	Compare two cities	Select 2 cities	Comparison graph + tables	Pass
TC08	Average AQI	Two cities selected	Correct avg values shown	Pass

6.5 Error Detection and Fixing

During testing, some errors were identified:

- Column name mismatch (AQI vs aqi)
- Graph not displaying initially
- Database connection issues

These issues were fixed by:

- Correcting column names
- Debugging Plotly integration
- Fixing database queries

6.6 Result of Testing

After performing all tests, it was observed that:

- All modules are functioning correctly
- The system provides accurate AQI data
- The user interface is responsive
- The graph and table display properly

The system is stable and ready for use.

6.7 Summary

This chapter described the testing process of the system. Various testing methods were used to ensure that the application works correctly and efficiently.

All identified issues were resolved, and the system was successfully tested. The final application performs as expected and meets all user requirements.

Chapter 7: Results & Discussion

7.1 Introduction

This chapter presents the results obtained from the Air Quality Analysis and Visualization Dashboard. It explains how the system performs in real-time and how the output is displayed to the user.

The results include both tabular and graphical representations of AQI data, which help users understand air pollution trends effectively.

7.2 System Output

The system successfully generates output in multiple formats to provide a clear and detailed understanding of air quality data.

- **Tabular Format:**

Displays AQI values along with city and date information. In comparison mode, the system displays two separate tables side-by-side for better analysis of both cities.

- **Graphical Format:**

Displays AQI trends using interactive graphs created with Plotly. In comparison mode, the graph shows multiple lines representing different cities.

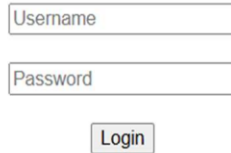
- **Comparative Output:**

The system allows users to compare AQI levels between two cities. It calculates and displays the average AQI values for each city and provides a result indicating which city is more polluted.

These outputs help users analyze air quality data in a simple, interactive, and more informative manner.

7.3 Login Page Result

Login

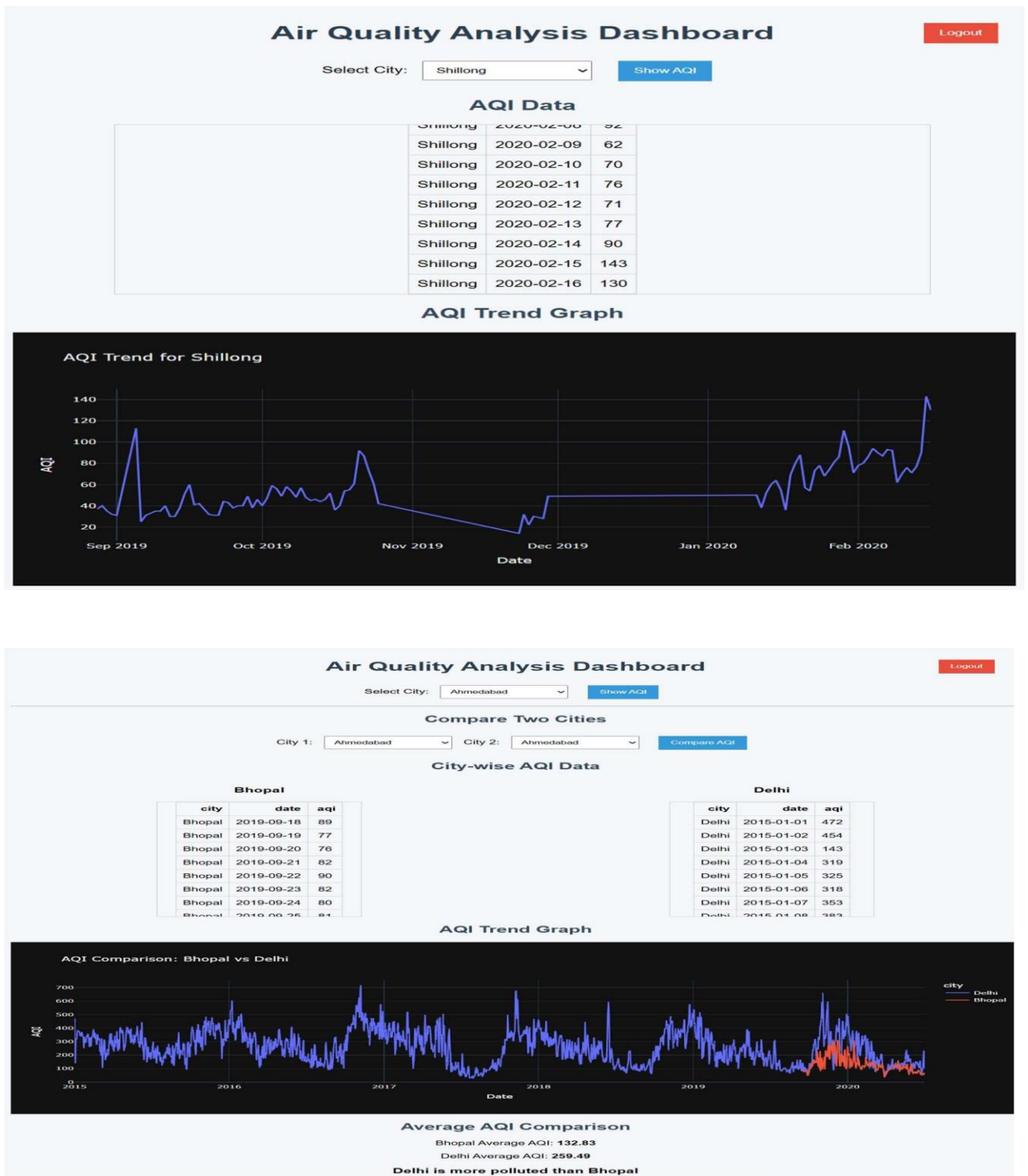


A login form with two text input fields. The first field is labeled 'Username' and the second field is labeled 'Password'. Below the password field is a button labeled 'Login'.

The login page allows users to enter their credentials.
Only valid users can access the dashboard.

- Username and password fields are provided
- Invalid login shows an error message
- Successful login redirects to dashboard

7.4 Dashboard Result



The dashboard is the main interface of the system.

It includes:

- City selection dropdown
- Button to fetch AQI data
- AQI data table
- Graph display
- Logout button

The design is clean and user-friendly.

7.5 AQI Data Table Result

AQI Data			
	city	date	aqi
	Delhi	2015-01-01	472
	Delhi	2015-01-02	454
	Delhi	2015-01-03	143
	Delhi	2015-01-04	319
	Delhi	2015-01-05	325
	Delhi	2015-01-06	318
	Delhi	2015-01-07	353
	Delhi	2015-01-08	383

Compare Two Cities

City 1:

Ahmedabad

City 2:

Ahmedabad

Compare AQI

City-wise AQI Data

Bhopal

city	date	aqi
Bhopal	2019-09-18	89
Bhopal	2019-09-19	77
Bhopal	2019-09-20	76
Bhopal	2019-09-21	82
Bhopal	2019-09-22	90
Bhopal	2019-09-23	82
Bhopal	2019-09-24	80
Bhopal	2019-09-25	81

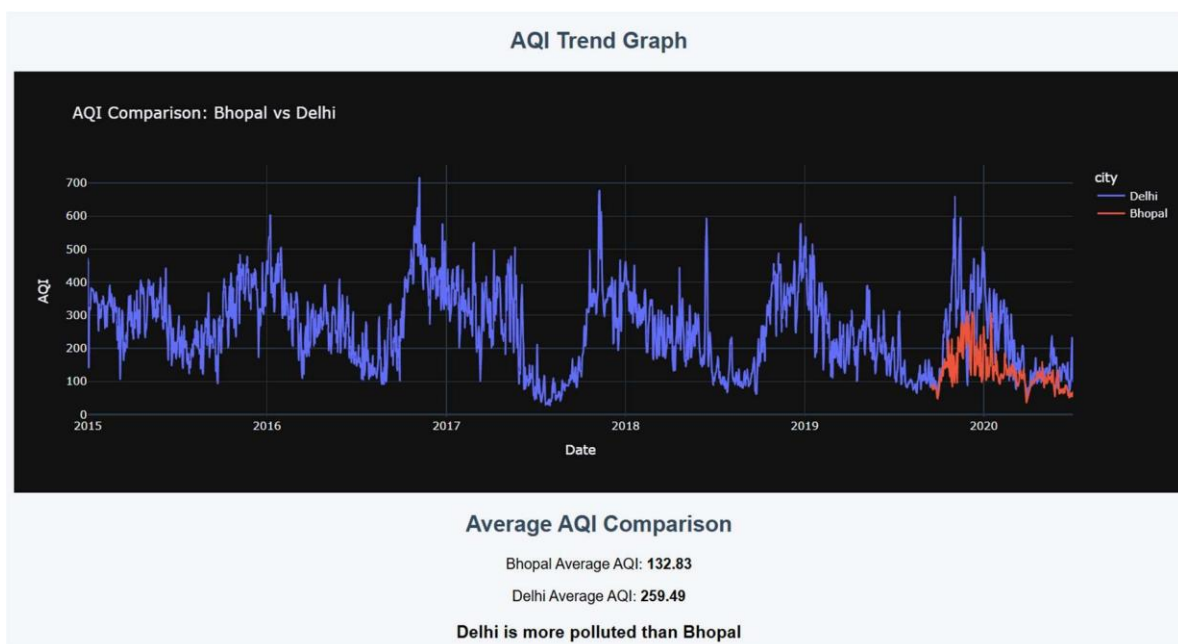
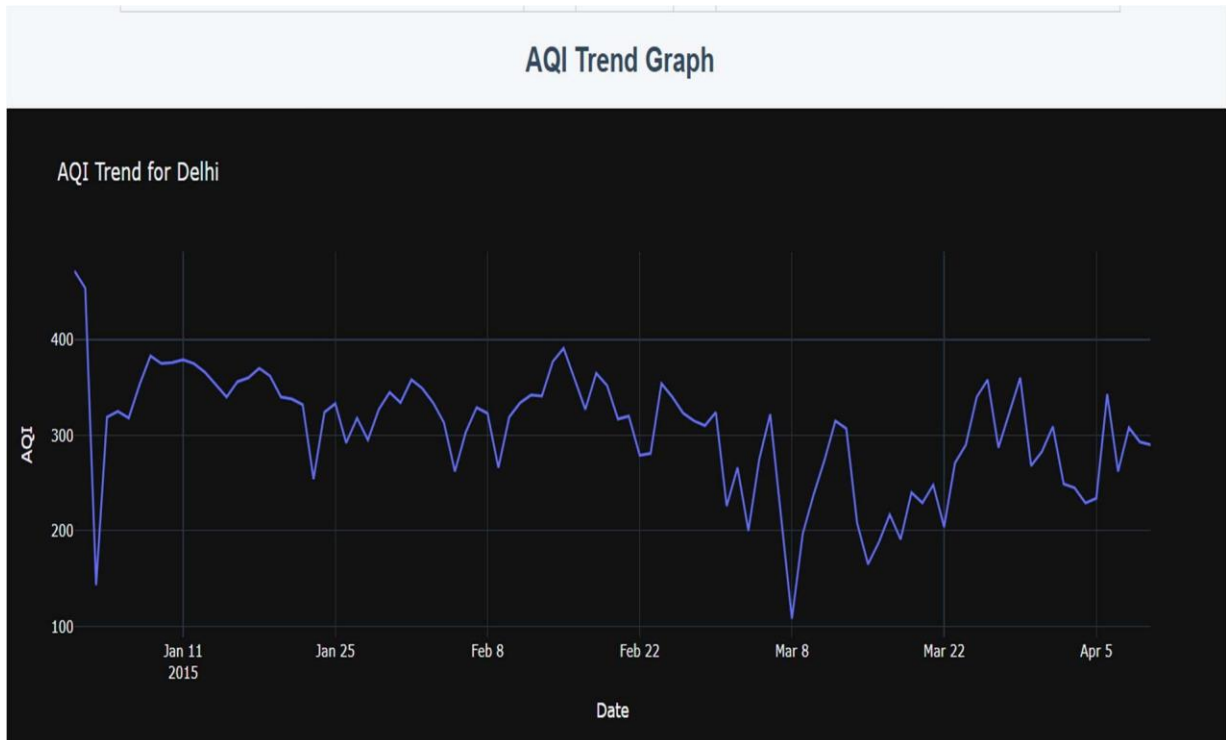
Delhi

city	date	aqi
Delhi	2015-01-01	472
Delhi	2015-01-02	454
Delhi	2015-01-03	143
Delhi	2015-01-04	319
Delhi	2015-01-05	325
Delhi	2015-01-06	318
Delhi	2015-01-07	353
Delhi	2015-01-08	383

The table displays detailed AQI data.

- Shows city, date, and AQI values
- Scrollable for better usability
- Helps users view exact data

7.6 AQI Graph Result



The graph shows AQI trends over time.

- Generated using Plotly
- Interactive and easy to understand
- Displays AQI trends for a single city or multiple cities in comparison mode
- Helps visualize pollution patterns and differences between cities
- Provides a clear view of air quality variations over time

7.7 Discussion

The developed system provides an effective way to analyze and visualize air quality data.

Key observations:

- Graphs help identify pollution trends easily
- Table provides detailed numerical data
- System is simple and easy to use
- Users can quickly select cities and view results

The combination of visualization and structured data improves understanding of AQI levels.

7.8 Advantages of the System

- Easy to use interface
- Interactive data visualization
- Quick data retrieval
- Helps in understanding air pollution trends
- Lightweight and efficient

7.9 Limitations of the System

- Limited to selected cities
- Uses static dataset (not real-time)
- Basic level visualization

- Limited filtering options

7.10 Summary

This chapter presented the results and discussion of the project. The system successfully displays AQI data in both tabular and graphical formats.

The results show that the dashboard is effective in analyzing air quality data and providing meaningful insights to users.

Chapter 8 : Conclusion & Future Scope

8.1 Introduction

This chapter presents the conclusion of the Air Quality Analysis and Visualization Dashboard project and discusses possible future improvements. It summarizes the overall work done and highlights the effectiveness of the developed system.

8.2 Conclusion

The Air Quality Analysis and Visualization Dashboard was successfully developed using Flask, MySQL, Pandas, and Plotly. The main objective of the project was to analyze AQI data and present it in a simple and understandable format through a web-based dashboard.

The system allows users to log in, select a city, and view AQI data in both tabular and graphical formats. It also enables users to compare AQI levels between two cities using interactive graphs and side-by-side tables. The system calculates and displays average AQI values, making it easier to understand pollution trends and differences between cities.

The integration of multiple technologies such as Flask for backend, MySQL for database management, Pandas for data processing, and Plotly for visualization ensured smooth functioning of the system. The application is user-friendly and provides accurate results based on the selected data.

Throughout the development process, several challenges were encountered, such as database connection issues, column name mismatches, and graph display problems. These issues were resolved through debugging and testing, which improved the overall performance of the system.

The project also helped in understanding important concepts such as web development, database connectivity, data analysis, and visualization techniques. It provided practical experience in building a real-world application.

Overall, the system meets all the objectives and provides an efficient solution for analyzing and visualizing air quality data.

8.3 Achievements of the Project

The following achievements were accomplished during the project:

- Successfully developed a working web-based dashboard
- Implemented user login functionality
- Integrated MySQL database with Flask
- Processed data using Pandas
- Created interactive graphs using Plotly
- Designed a simple and clean user interface
- Displayed AQI data in both table and graphical formats

These achievements demonstrate the successful completion of the project.

8.4 Limitations of the Project

Despite successful implementation, the system has some limitations:

- The system uses static data instead of real-time data
- Limited number of cities are included
- Basic level visualization features
- No advanced filtering options
- Security features are basic

These limitations can be improved in future versions of the system.

8.5 Future Scope

The project can be further enhanced with the following features:

- Integration of real-time AQI data from APIs
- Addition of more cities and datasets
- Implementation of advanced filters such as date range and pollutants
- Improved user interface using frameworks like Bootstrap or React
- Enhanced security using password encryption and session management
- Deployment of the application on cloud platforms
- Mobile-friendly design for better accessibility

These improvements will make the system more advanced and useful.

8.6 Practical Applications

The developed system can be useful in various real-world scenarios:

- Helps users monitor air quality in different cities
- Useful for environmental awareness
- Can assist researchers in analyzing pollution trends
- Helps authorities in decision-making related to pollution control

8.7 Summary

This chapter concluded the project by summarizing the overall work and highlighting its achievements. The system successfully meets its objectives and provides a user-friendly platform for analyzing AQI data.

The future scope suggests that the system can be expanded and improved further to make it more powerful and scalable.

Chapter 9: References

The following sources were referred to during the development of the project:

1. Python Software Foundation. *Python Documentation*. Available at:
<https://docs.python.org/>
2. Pallets Projects. *Flask Documentation*. Available at:
<https://flask.palletsprojects.com/>
3. Oracle Corporation. *MySQL Documentation*. Available at:
<https://dev.mysql.com/doc/>
4. Wes McKinney. *Pandas Documentation*. Available at:
<https://pandas.pydata.org/docs/>
5. Plotly Technologies Inc. *Plotly Documentation*. Available at:
<https://plotly.com/python/>
6. W3Schools. *HTML Tutorial*. Available at: <https://www.w3schools.com/html/>
7. W3Schools. *CSS Tutorial*. Available at: <https://www.w3schools.com/css/>
8. Mozilla Developer Network. *Web Development Guide*. Available at:
<https://developer.mozilla.org/>
9. Stack Overflow. *Programming Q&A Platform*. Available at:
<https://stackoverflow.com/>
10. GeeksforGeeks. *Python and Web Development Tutorials*. Available at:
<https://www.geeksforgeeks.org/>
11. Kaggle. *Datasets for Air Quality Analysis*. Available at: <https://www.kaggle.com/>
12. Central Pollution Control Board (CPCB), India. *Air Quality Data*. Available at:
<https://cpcb.nic.in/>

13. World Health Organization (WHO). *Air Quality Guidelines*. Available at:
<https://www.who.int/>
14. U.S. Environmental Protection Agency (EPA). *Air Quality Index (AQI)*. Available
at: <https://www.epa.gov/aqi>
15. IBM. *Data Visualization Concepts*. Available at: <https://www.ibm.com/analytics>
16. Tutorialspoint. *Flask Tutorial*. Available at: <https://www.tutorialspoint.com/flask/>
17. Real Python. *Flask Web Development Tutorials*. Available at:
<https://realpython.com/>
18. Programiz. *Python Programming Tutorials*. Available at:
<https://www.programiz.com/python-programming>
19. DataCamp. *Data Analysis and Visualization*. Available at:
<https://www.datacamp.com/>
20. GitHub. *Open-source Code References*. Available at: <https://github.com/>

Chapter 10: Appendices

10.1 web_app

10.1.1 templates

App.py

```
from flask import Flask, render_template, request, redirect, url_for
```

```
import mysql.connector
```

```
import pandas as pd
```

```
import plotly.express as px
```

```
app = Flask(__name__)
```

```
conn = mysql.connector.connect(  
    host="localhost",  
    user="root",  
    password="Smruti@2004",  
    database="aqi_project"  
)
```

```
# ♦ Home → Redirect to login
```

```
@app.route("/")
```

```
def home():
```

```
    return redirect(url_for("login"))
```

```
# ♦ Login Page
```

```
@app.route("/login", methods=["GET", "POST"])
```

```
def login():  
    if request.method == "POST":  
        username = request.form["username"]  
        password = request.form["password"]  
  
        cursor = conn.cursor()  
        cursor.execute("SELECT * FROM users WHERE username=%s AND  
            password=%s", (username, password))  
        user = cursor.fetchone()  
  
        if user:  
            return redirect(url_for("dashboard"))  
        else:  
            return "Invalid Login"  
  
    return render_template("login.html")  
  
# ◆ Logout  
@app.route("/logout")  
def logout():  
    return redirect(url_for("login"))  
  
# ◆ Dashboard  
@app.route("/dashboard", methods=["GET", "POST"])  
def dashboard():
```

```
cursor = conn.cursor()

cursor.execute("SELECT DISTINCT city FROM air_quality_data")

cities = [row[0] for row in cursor.fetchall()]


city = None

data = None

graph_html = None


if request.method == "POST":

    city = request.form["city"]


    query = """
SELECT city, date, aqi
FROM air_quality_data
WHERE city = %s
ORDER BY date
LIMIT 100
    """

    data = pd.read_sql(query, conn, params=[city])


    if not data.empty:

        fig = px.line(data, x="date", y="aqi", title=f"AQI Trend for {city}")
```

```
fig.update_layout(  
    xaxis_title="Date",  
    yaxis_title="AQI",  
    template="plotly_dark"  
)  
graph_html = fig.to_html(full_html=False)  
  
return render_template(  
    "index.html",  
    cities=cities,  
    tables=data.to_html(index=False) if data is not None and not data.empty else None,  
    city=city,  
    graph=graph_html  
)  
  
@app.route("/compare", methods=["POST"])  
def compare():  
  
    cursor = conn.cursor()  
    cursor.execute("SELECT DISTINCT city FROM air_quality_data")  
    cities = [row[0] for row in cursor.fetchall()]  
  
    city1 = request.form["city1"]  
    city2 = request.form["city2"]  
  
    query = ""
```

```
SELECT city, date, aqi
FROM air_quality_data
WHERE city IN (%s, %s)
ORDER BY date
""""
```

```
data = pd.read_sql(query, conn, params=[city1, city2])
```

```
# Split data for each city
```

```
data1 = data[data["city"] == city1]
```

```
data2 = data[data["city"] == city2]
```

```
table1 = data1.to_html(index=False)
```

```
table2 = data2.to_html(index=False)
```

```
# ◆ Average AQI
```

```
avg_data = data.groupby("city")["aqi"].mean().reset_index()
```

```
avg1 = round(avg_data[avg_data["city"] == city1]["aqi"].values[0], 2)
```

```
avg2 = round(avg_data[avg_data["city"] == city2]["aqi"].values[0], 2)
```

```
# ◆ Graph
```

```
fig = px.line(data, x="date", y="aqi", color="city",
```

```
title=f"AQI Comparison: {city1} vs {city2}")
```

```
fig.update_layout(
```

```
        xaxis_title="Date",
        yaxis_title="AQI",
        template="plotly_dark"
    )

    graph_html = fig.to_html(full_html=False)

    # ◆ Result

    if avg1 > avg2:
        result = f"{city1} is more polluted than {city2}"
    else:
        result = f"{city2} is more polluted than {city1}"

    return render_template(
        "index.html",
        cities=cities,
        table1=table1,
        table2=table2,
        graph=graph_html,
        city1=city1,
        city2=city2,
        avg1=avg1,
        avg2=avg2,
        result=result
    )
```

```
if __name__ == "__main__":  
    app.run(debug=True)
```

10.1.2 index.html

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
<title>Air Quality Dashboard</title>  
  
<style>  
  
    body{  
        font-family: Arial;  
        background:#f4f6f9;  
        text-align:center;  
    }  
  
    h1{  
        color:#2c3e50;  
        margin-top:30px;  
    }  
  
    h2{  
        color:#34495e;
```

```
}
```

```
select,button{  
    padding:8px;  
    margin:10px;  
}
```

```
button{  
background:#3498db;  
    color:white;  
    border:none;  
padding:10px 15px;  
    cursor:pointer;  
}
```

```
button:hover{  
background:#2980b9;  
}
```

```
table{  
    margin:auto;  
border-collapse:collapse;  
}
```

```
th,td{  
border:1px solid #ccc;
```

```
padding:8px;  
}
```

```
.table-container{  
    height:300px;  
    overflow-y:scroll;  
    margin:auto;  
    width:80%;  
border:1px solid #ccc;  
}
```

```
.tables{  
    display:flex;  
justify-content:space-around;  
    gap:20px;  
}
```

```
.tables div{  
    width:45%;  
}
```

```
.logout{  
position:absolute;  
    top:20px;  
    right:30px;  
}
```

```
.logout button{  
background:#e74c3c;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="logout">
```

```
<a href="/logout">
```

```
<button>Logout</button>
```

```
</a>
```

```
</div>
```

```
<h1>Air Quality Analysis Dashboard</h1>
```

```
<form method="POST" action="/dashboard">
```

```
<label>Select City:</label>
```

```
<select name="city">
```

```
{% for c in cities %}
```

```
<option value="{{c}}" {% if city==c %}selected{% endif %}>{{c}}</option>
```

```
{% endfor %}
```

```
</select>
```

```
<button type="submit">Show AQI</button>
```

```
</form>
```

```
<hr>
```

```
<h2>Compare Two Cities</h2>
```

```
<form method="POST" action="/compare">
```

```
<label>City 1:</label>
```

```
<select name="city1">
```

```
{% for c in cities %}
```

```
<option value="{{c}}">{{c}}</option>
```

```
{% endfor %}
```

```
</select>
```

```
<label>City 2:</label>
```

```
<select name="city2">
```

```
{% for c in cities %}
```

```
<option value="{{c}}">{{c}}</option>
```

```
{% endfor %}
```

```
</select>
```

```
<button type="submit">Compare AQI</button>
```

```
</form>
```

```
{% if tables %}
```

```
<h2>AQI Data</h2>
```

```
<div class="table-container">
```

```
{{ tables|safe }}
```

```
</div>
```

```
{% endif %}
```

```
{% if table1 and table2 %}
```

```
<h2>City-wise AQI Data</h2>
```

```
<div class="tables">
```

```
<div>
```

```
<h3>{{ city1 }}</h3>
```

```
<div class="table-container">
```

```
{{ table1|safe }}
```

```
</div>
```

```
</div>
```

```
<div>
```

<h3>{{ city2 }}</h3>

<div class="table-container">

{{ table2|safe }}

</div>

</div>

</div>

{% endif %}

{% if graph %}

<h2>AQI Trend Graph</h2>

{{ graph|safe }}

{% endif %}

{% if avg1 and avg2 %}

<h2>Average AQI Comparison</h2>

<p>{{ city1 }} Average AQI: {{ avg1 }}</p>

<p>{{ city2 }} Average AQI: {{ avg2 }}</p>

<h3>{{ result }}</h3>

{% endif %}

</body>

</html>

10.1.3 login.html

```
<!DOCTYPE html>

<html>

<head>

<title>Login</title>

</head>


<body style="text-align: center; margin-top:100px;">


<h2>Login</h2>


<form method="POST">

<input type="text" name="username" placeholder="Username" required><br><br>

<input type="password" name="password" placeholder="Password" required><br><br>

<button type="submit">Login</button>

</form>


</body>

</html>
```

10.2 Database

10.2.1 queries.sql

-- Count total rows

```
SELECT COUNT(*)  
FROM air_quality_data;
```

-- Average AQI by city

```
SELECT city, AVG(aqi) AS average_aqi  
FROM air_quality_data  
GROUP BY city;
```

-- Delhi vs Shillong comparison

```
SELECT city, AVG(aqi) AS avg_aqi  
FROM air_quality_data  
WHERE city IN ('Delhi', 'Shillong')  
GROUP BY city;
```

-- Pollutant comparison

```
SELECT city,  
AVG(pm25) AS avg_pm25,  
AVG(pm10) AS avg_pm10,  
AVG(no2) AS avg_no2,  
AVG(so2) AS avg_so2,  
AVG(co) AS avg_co,  
AVG(o3) AS avg_o3
```

```
FROM air_quality_data

GROUP BY city;


-- Count total rows

SELECT COUNT(*)

FROM air_quality_data;


-- Average AQI by city

SELECT city, AVG(aqi) AS average_aqi

FROM air_quality_data

GROUP BY city;


-- Delhi vs Shillong comparison

SELECT city, AVG(aqi) AS avg_aqi

FROM air_quality_data

WHERE city IN ('Delhi', 'Shillong')

GROUP BY city;


-- Pollutant comparison

SELECT city,

AVG(pm25) AS avg_pm25,

AVG(pm10) AS avg_pm10,

AVG(no2) AS avg_no2,

AVG(so2) AS avg_so2,

AVG(co) AS avg_co,

AVG(o3) AS avg_o3
```

```
FROM air_quality_data
```

```
GROUP BY city;
```

```
SELECT * FROM air_quality_data;
```

```
SELECT DISTINCT city FROM air_quality_data;
```

```
SELECT *
```

```
FROM air_quality_data
```

```
LIMIT 5;
```

```
SELECT Date, AQI
```

```
FROM air_quality_data
```

```
WHERE City='Delhi'
```

```
LIMIT 5;
```

```
CREATE TABLE cities (
```

```
city_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
city_name VARCHAR(50)
```

```
);
```

```
select*from cities;
```

```
SELECT * FROM air_quality_data;
```

```
describe air_quality_data;
```

```
ALTER TABLE air_quality_data
```

```
ADD city_id INT;
```

```
ALTER TABLE air_quality_data
```

```
ADD FOREIGN KEY (city_id) REFERENCES cities(city_id);
```

```
CREATE TABLE users (
```

```
user_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
username VARCHAR(50),
```

```
password VARCHAR(50)
```

```
);
```

```
select*from users;
```

```
INSERT INTO users (username, password)
```

```
VALUES ('smruti', '2004');
```

```
INSERT INTO cities (city_name) VALUES
```

```
('Delhi'),('Mumbai'),('Chennai'),('Shillong');
```

10.3 python_analysis

10.3.1 aqi_analysis.py

```
import pandas as pd
```

```
# Load dataset
```

```
data = pd.read_csv("../data/city_day.csv")
```

```
# Filter Delhi and Shillong

cities = data [
data["City"].str.contains("Delhi", case=False) |
data["City"].str.contains("shill", case=False)
]

print(cities["City"].unique())

print(cities.head())

# Save filtered data
cities.to_csv("delhi_shillong_data.csv", index=False)

print("Filtered data saved successfully!")
print(cities[cities["City"].str.contains("shill", case=False)].head())

import matplotlib.pyplot as plt

# Calculate average AQI for each city
avg_aqi = cities.groupby("City")["AQI"].mean()

print(avg_aqi)

# ----- BAR GRAPH -----

plt.figure()
```

```
avg_aqi.plot(kind="bar")

plt.title("Average AQI Comparison")
plt.xlabel("City")
plt.ylabel("Average AQI")
plt.xticks(rotation=0)

plt.savefig("../Charts/average_aqi_comparison.png")

plt.show()

# ----- TREND GRAPH -----

# Convert Date column to datetime
cities["Date"] = pd.to_datetime(cities["Date"])

# Sort data
cities = cities.sort_values("Date")

plt.figure()

for city in cities["City"].unique():
    city_data = cities[cities["City"] == city]
    plt.plot(city_data["Date"], city_data["AQI"], label=city)
```

```
plt.title("AQI Trend Comparison")

plt.xlabel("Date")

plt.ylabel("AQI")

plt.legend()

plt.savefig("../Charts/aqi_trend_comparison.png")

plt.show()

# ----- PM2.5 COMPARISON -----

pm25_avg = cities.groupby("City")["PM2.5"].mean()

plt.figure()

pm25_avg.plot(kind="bar")

plt.title("Average PM2.5 Comparison")

plt.xlabel("City")

plt.ylabel("PM2.5 Level")

plt.xticks(rotation=0)

plt.savefig("../Charts/pm25_comparison.png")

plt.show()

# ----- TOP POLLUTANTS IN DELHI -----
```

```
delhi_data = cities[cities["City"].str.contains("Delhi", case=False)]
```

```
pollutants = ["PM2.5", "PM10", "NO2", "SO2", "CO", "O3"]
```

```
pollution_avg = delhi_data[pollutants].mean()
```

```
plt.figure()
```

```
pollution_avg.plot(kind="bar")
```

```
plt.title("Average Pollutant Levels in Delhi")
```

```
plt.xlabel("Pollutant")
```

```
plt.ylabel("Average Level")
```

```
plt.xticks(rotation=0)
```

```
plt.savefig("../Charts/delhi_pollutants.png")
```

```
plt.show()
```

10.3.2 load_to_mysql.py

```
import mysql.connector
```

```
import pandas as pd
```

```
conn = mysql.connector.connect(
```

```
host="localhost",  
user="root",  
password="Smruti@2004",  
database="aqi_project"  
)  
  
cursor = conn.cursor()  
  
data = pd.read_csv("../Data/city_day.csv")  
  
# Convert NaN to None (MySQL NULL)  
data = data.replace({pd.NA: None})  
data = data.where(pd.notnull(data), None)  
  
for _, row in data.iterrows():  
    sql = ""  
  
    INSERT INTO air_quality_data  
(city, date, pm25, pm10, no2, so2, co, o3, aqi)  
VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s)  
    ""  
  
    values = (  
        row["City"] if pd.notna(row["City"]) else None,  
        row["Date"] if pd.notna(row["Date"]) else None,  
        row["PM2.5"] if pd.notna(row["PM2.5"]) else None,  
        row["PM10"] if pd.notna(row["PM10"]) else None,
```

```

row["NO2"] if pd.notna(row["NO2"]) else None,

row["SO2"] if pd.notna(row["SO2"]) else None,

row["CO"] if pd.notna(row["CO"]) else None,

row["O3"] if pd.notna(row["O3"]) else None,

row["AQI"] if pd.notna(row["AQI"]) else None

)

```

```

cursor.execute(sql, values)

```

```

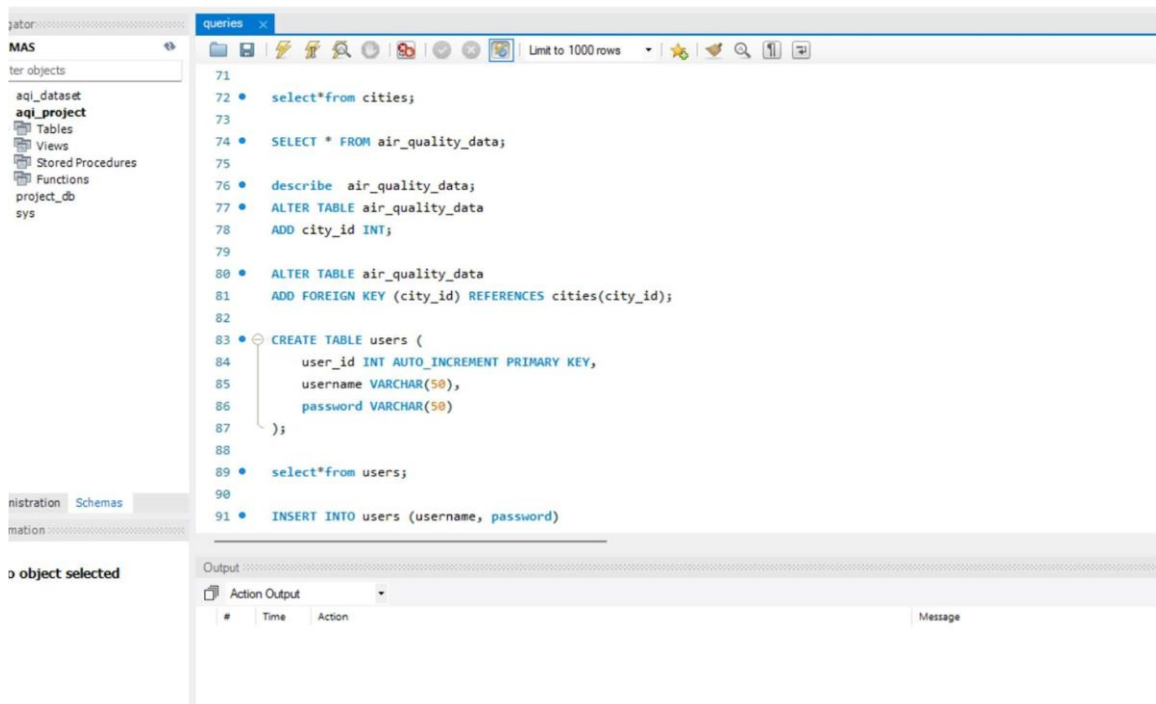
conn.commit()

```

```

print("Data inserted successfully")

```



```

EXPLORER
AIR_QUALITY_ANALYTICS_SYST...
  Charts
  Dashboard
  aqi dashboard graph.pdf
  AQI power Bi.pbix
  Data
  Database
  Python_analysis
  Report
  web_app
  statics
  templates
  index.html
  login.html
  app.py

web_app > app.py > ...
1 from flask import Flask, render_template, request, redirect, url_for
2 import mysql.connector
3 import pandas as pd
4 import plotly.express as px
5
6 app = Flask(__name__)
7
8 conn = mysql.connector.connect(
9     host="localhost",
10    user="root",
11    password="Smrutid2004",
12    database="aqi_project"
13 )
14
15 # Home -> Redirect to login
16 @app.route("/")
17 def home():
18     return redirect(url_for("login"))
19
20 # Login Page
21 @app.route("/login", methods=["GET", "POST"])

```

DEBUG CONSOLE PROBLEMS 1 TERMINAL OUTPUT PORTS SPELL CHECKER 1

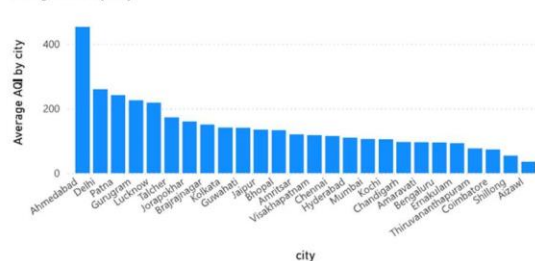
```

* Debugger is active!
* Debugger PIN: 315-934-384
* Detected change in 'D:\Air_Quality_Analytics_System\web_app\app.py', reloading
* Restarting with watchdog (windowsapi)
* Debugger is active!

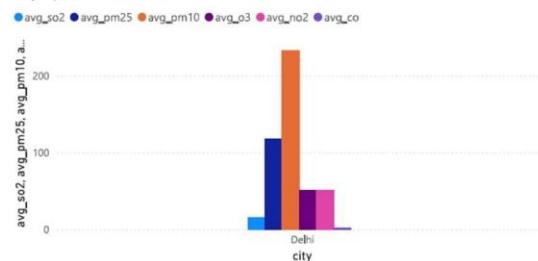
```

Air Quality Analysis Dashboard

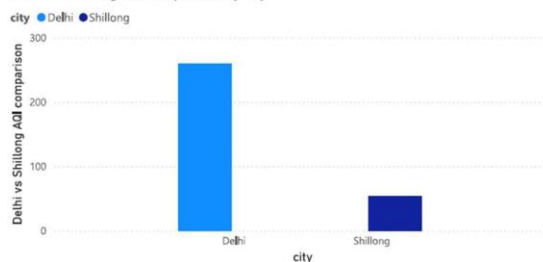
Average AQI by city



Major pollutants in Delhi



Delhi vs Shillong AQI comparison by city



Pollutant Contribution by City

